AWS lamda

AWS Lambda is a compute service that runs code without the need to manage servers. Your code runs, scaling up and down automatically, with pay-per-use pricing..

you can use Lambda for:

- **File processing**: Process files automatically when uploaded to Amazon Simple Storage Service.
- **Long-running workflows:** Use durable Lambda functions to build stateful, multi-step workflows that can run for up to one year. Perfect for order processing, approval workflows, human-in-the-loop processes, and complex data pipelines that need to remember their progress.
- **Database operations and integration examples**: Respond to database changes and automate data workflows.
- **Scheduled and periodic tasks**: Run automated operations on a regular schedule using EventBridge..
- **Stream processing**: Process real-time data streams for analytics and monitoring.

Functions and Durable functions:

Lamda function: this is a pice of code run in response to pice of event run for up to 15 minutes and are ideal for event-driven tasks like processing API requests, handling file uploads, or responding to database changes

Characterstics

1. Stateless by default (each invocation is independent).
2. Automatically scales.

3. You pay only for execution time.

Example flow:

1. A file is uploaded to an S3 bucket.
2. Lambda is triggered.
3. Lambda processes the file and exits.

Use cases:

- API backends
- File processing
- Database triggers
- Scheduled tasks

Durable functions: They can execute for up to one year, automatically checkpointing their progress so they resume reliably after failures. The term **Durable Functions** is most commonly associated with **Azure Functions**, where workflows can maintain state across multiple executions.

Characteristics of Durable Workflows:

- Can maintain workflow state.
- Support long-running processes.
- Handle retries and error recovery.
- Coordinate multiple Lambda functions.

Example workflow:

1. Lambda validates an order.
2. Another Lambda processes payment.
3. Another Lambda updates inventory.
4. Another Lambda sends confirmation email.
5. Workflow state is managed by Step Functions.

How lamda works

1. Lambda functions are the basic building blocks you use to build Lambda applications.
2. To write functions, it's essential to understand the core concepts and components that make up the Lambda programming model

Lambda functions and function handlers

Functions are the basic building blocks to bulid the application . . Function handlers are the entry point for event objects that your Lambda function code processes.

Lambda execution environment and runtimes

- Lambda execution environments manage the resources required to run your function

Events and triggers

Other AWS services can invoke your functions in response to specific events.

Lambda permissions and roles

1. Control who can access your functions and what other AWS services your functions
2. Permissions to other users and AWS needs to access your function

Permissions for functions to access other AWS resources

Lambda functions often need to access other AWS resources and perform actions on them.

How it works

1. You create an **IAM role**.
2. Attach policies that grant access to required AWS services.
3. Assign the role to the Lambda function.
4. When the function runs, it temporarily assumes that role and uses its permissions.
5. **Example: Lambda accessing DynamoDB**
6. Suppose your Lambda function needs to read and write data in a DynamoDB table.
7. **IAM Policy:**
8.
```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "dynamodb:GetItem",
        "dynamodb:PutItem",
        "dynamodb:UpdateItem"
      ],
      "Resource": "arn:aws:dynamodb:region:account-id:table/Orders"
    }
  ]
}
```
9. Attach this policy to the Lambda execution role.

Permissions for other users and resources to access your function

Permissions for other users and resources to access your Lambda function are controlled using a **resource-based policy** attached to the Lambda function.

This is different from the **execution role**, which controls what the function can access.

Permission Type	Purpose
Execution Role (IAM Role)	What the Lambda function can access (S3, DynamoDB, etc.)
Resource-Based Policy	Who or what can invoke the Lambda function

Common AWS services that invoke Lambda:

- Amazon S3

- Amazon EventBridge
- Amazon API Gateway
- Amazon SNS

Allowing Another AWS Account

You can grant another AWS account permission to invoke your function:

```
{
  "Effect": "Allow",
  "Principal": {
    "AWS": "arn:aws:iam::123456789012:root"
  },
  "Action": "lambda:InvokeFunction",
  "Resource": "arn:aws:lambda:us-east-1:account-
id:function:MyFunction"
}
```

This enables cross-account access.

Create a Lambda function

1. Sign in to the AWS Management Console and open the Lambda console at https://console.aws.amazon.com/lambda/.
2. Choose **Create function**.
3. Select **Author from scratch**.
4. In the **Basic information** section:
- For **Function name**, enter a function name (for example, `DateTimeFunction`). Note the name of the function, you'll need it in step 15 of Step 2: Create an Amazon Bedrock agent.
- For **Runtime**, select **Python 3.9** (or your preferred version).
- For **Architecture**, leave unchanged.
- In **Permissions**, select **Change default execution role** and then select **Create a new role with basic Lambda permissions**.
5. Choose **Create function**.
6. In **Function overview**, under **Function ARN**, note the Amazon Resource Name (ARN) for the function. You need it for step 24 of Step 2: Create an lamdahandler event
7. In the **Code** tab, replace the existing code with the following:

```
8. # Copyright Amazon.com, Inc. or its affiliates. All Rights Reserved.
```

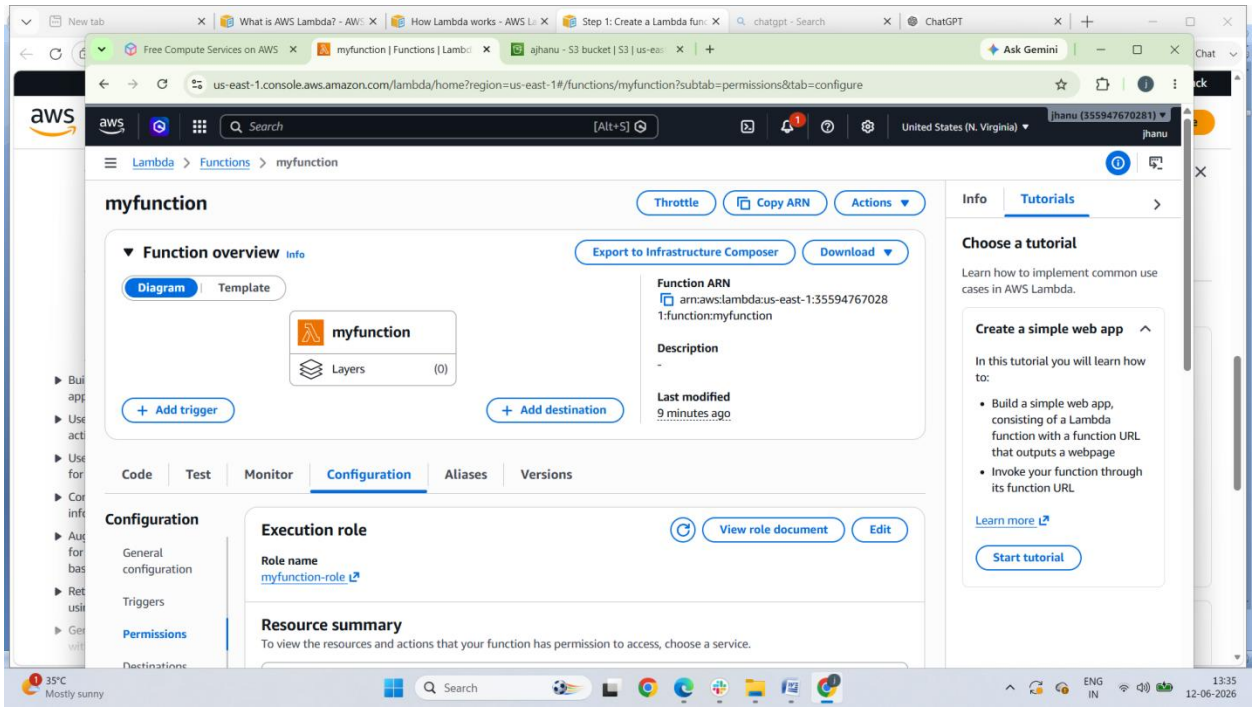
```
9. # SPDX-License-Identifier: Apache-2.0
```

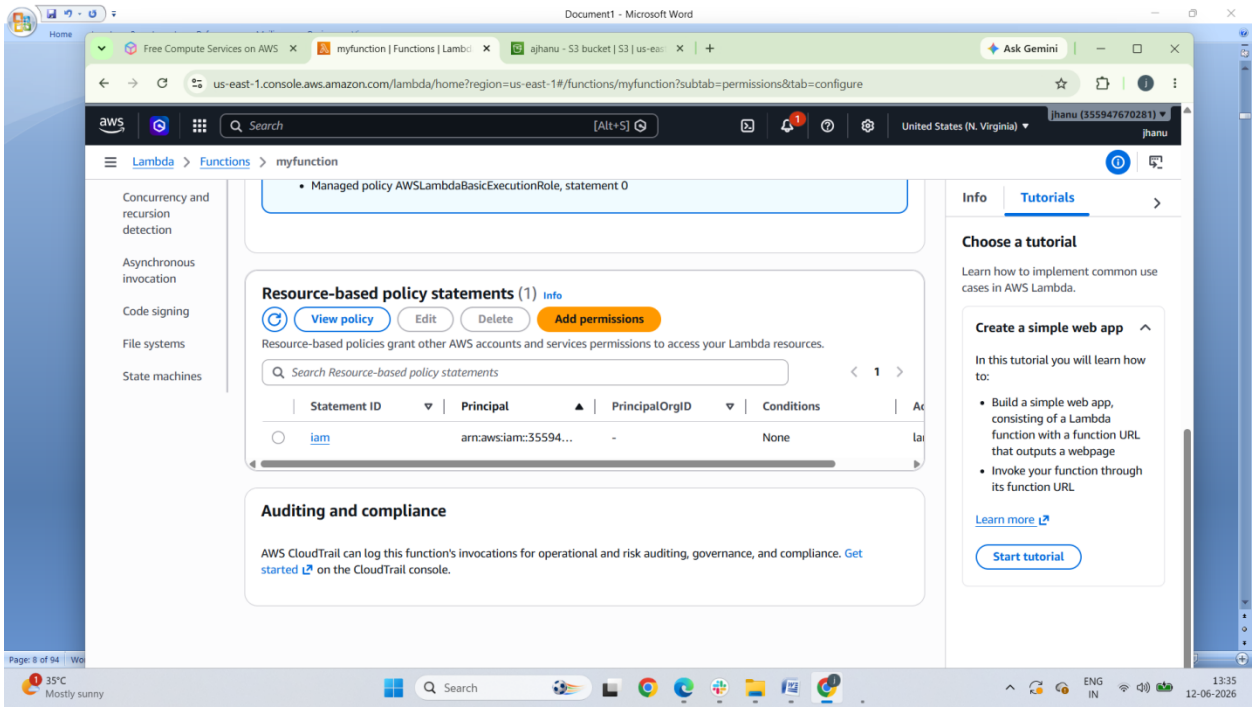
```
    import json



def lambda_handler(event, context):

    print("Received event:", json.dumps(event))



    return {

        "statusCode": 200,

        "body": json.dumps({

            "message": "Hello from Lambda!"

        })

    }
```

10. Choose **Deploy** to deploy your function.
11. Choose the **Configuration** tab.
12. Choose **Permissions**.
13. Under **Resource-based policy statements**, choose **Add permissions**.
14. In **Edit policy statement**, do the following:
    a. Choose **AWS service**
    b. In **Service** select **Other**.
    c. For **Statement ID**, enter a unique identifier (for example, `iam`)
    d. For **Principal**, enter arn:aws:iam::35547672801:root
    e. For **Source ARN**, enter `arn:aws:bedrock:region:AWS account ID:agent/*`

    Replace `region` with AWS Region that you are using, such as `us-east-1`. Replace `AWS account ID` your AWS account Id.

    f. For **Action**, select `lambda:InvokeFunction`.
15. Choose **Save**.





Amazon API Gateway

Amazon API Gateway is an AWS service for creating, publishing, maintaining, monitoring, and securing REST, HTTP, and WebSocket APIs at any scale

It acts as a **front door** for applications to access backend services such as:

- AWS Lambda functions
- EC2 applications
- Containers running on ECS/EKS
- HTTP endpoints
- Other AWS services

API Gateway creates RESTful APIs that:

- Are HTTP-based.
- Enable stateless client-server communication.
- Implement standard HTTP methods such as GET, POST, PUT, PATCH, and DELETE.

API Gateway Architecture

Client

↓

API Gateway

↓

Lambda / EC2 / ECS / Backend Service

Use cases of API gateway:

Amazon API Gateway is commonly used as a secure and scalable entry point for applications and services.

1. Serverless Applications

API Gateway receives HTTP requests and invokes AWS Lambda functions.

Flow:

Client → API Gateway → Lambda → Response

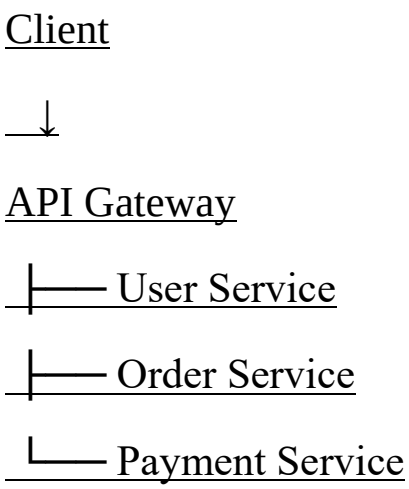
Example:

- User submits a form.
- API Gateway triggers a Lambda function.
- Lambda stores data in a database.

2. Microservices Architecture

API Gateway provides a single entry point to multiple backend services.

Flow:



Benefits:

- Centralized authentication
- Request routing
- Monitoring

3. Mobile Application Backends

Mobile apps can securely access backend services through API Gateway.

Example:

- User logs into a mobile app.
- API Gateway validates the user.
- Backend returns profile information.

4. Web Application APIs

Expose REST or HTTP APIs for web applications.

Examples:

- E-commerce product APIs
- Customer management systems
- Booking applications

5. Authentication and Authorization Layer

API Gateway can secure APIs using:

- IAM
- Amazon Cognito
- JWT tokens
- Lambda Authorizers

This avoids implementing authentication logic in every service.

6. Request Throttling and Rate Limiting

Protect backend services from excessive traffic.

Example:

- Limit clients to 100 requests per second.
- Prevent accidental or malicious overload.

7. Real-Time Applications

7.API Gateway supports WebSocket APIs.

Examples:

- Chat applications
- Live notifications
- Multiplayer gaming updates

8. API Management for External Partners

Expose APIs to third-party developers while controlling usage.

Features:

- API keys
- Usage plans
- Monitoring and analytics

9. Legacy System Integration

API Gateway can expose older systems as modern REST APIs.

Example:

```
Client → API Gateway → Legacy Application
```

This allows modernization without rewriting the entire backend.

10. Internal Enterprise APIs

Organizations use API Gateway to expose internal services securely.

Examples:

- Employee portals
- HR systems
- Internal reporting APIs

API Gateway Concepts:

Amazon API Gateway is built around a few core building blocks that define how APIs are created, deployed, secured, and executed.

1. API

An **API** is the top-level container.

- Represents a collection of endpoints and methods
- Example: `/users`, `/orders`

Types:

- REST API
- HTTP API
- WebSocket API

2.Resource

A **resource** is a path in your API.

Example:

```
/users
/users/{id}
```

Resources define the URL structure.

3. Method

A **method** defines the HTTP operation on a resource.

Common methods:

- GET → Read data
- POST → Create data
- PUT → Update data
- DELETE → Remove data

Example:

```
GET /users
POST /users
```

4. Integration

Defines **what backend the API connects to**.

Backends can be:

- AWS Lambda
- HTTP endpoint
- AWS service
- Mock integration

Example:

```
API Gateway → Lambda function
```

5.Stage

A **stage** is a deployment environment.

Examples:

- dev
- test
- prod

Each stage has its own URL:

```
/dev/users
/prod/users
```

6. Deployment

A **deployment** is a snapshot of your API configuration.

- Required before API becomes accessible
- Changes are not live until deployed to a stage

7. Endpoint Types

API Gateway supports:

- **Edge-optimized** → Global users (uses CloudFront)
- **Regional** → Users in specific region
- **Private** → Access only inside VPC

8. Authorizer (Security Layer)

Controls **who can access the API**.

Types:

- IAM authorization
- Lambda authorizer
- Amazon Cognito

9. API Key

Used to:

- Identify clients
- Track usage
- Apply rate limits

10. Usage Plan

Controls:

- Throttling limits (requests/sec)
- Quotas (requests/day/month)
- API key association

11. Throttling

Protects backend from overload.

Example:

- 100 requests/sec per user
- Burst limit handling

12. Mapping Template

Transforms request/response data.

Example:

- Convert JSON format
- Modify headers
- Change request structure before sending to backend

13. Caching

Improves performance by storing responses.

- Reduces backend calls
- Speeds up repeated requests

14. Logging & Monitoring

Integrated with:

- Amazon CloudWatch
- AWS X-Ray

Used for:

- Request tracing
- Error debugging
- Performance monitoring

Get started with the REST API console

Step 1: Create a Lambda function

To create a Lambda function

1. Sign in to the Lambda console at https://console.aws.amazon.com/lambda.
2. Choose **Create function**.
3. Under **Basic information**, for **Function name**, enter `my-function`.
4. For all other options, use the default setting.
5. Choose **Create function**.

Step 2: Create a REST API

To create a REST API

1. Sign in to the API Gateway console at https://console.aws.amazon.com/apigateway.
2. Do one of the following:

- To create your first API, for **REST API**, choose **Build**.
- If you've created an API before, choose **Create API**, and then choose **Build** for **REST API**.

3. For **API name**, enter `my-rest-api`.
4. (Optional) For **Description**, enter a description.
5. Keep **API endpoint type** set to **Regional**.
6. For **IP address type**, select **IPv4**.
7. Choose **Create API**.

Step 3: Create a Lambda proxy integration

To create a Lambda proxy integration

1. Select the `/` resource, and then choose **Create method**.
2. For **Method type**, select `ANY`.
3. For **Integration type**, select **Lambda**.
4. Turn on **Lambda proxy integration**.
5. For **Lambda function**, enter `my-function`, and then select your Lambda function.
6. Choose **Create method**.

Step 4: Deploy your API

To deploy your API

1. Choose **Deploy API**.
2. For **Stage**, select **New stage**.
3. For **Stage name**, enter `Prod`.
4. (Optional) For **Description**, enter a description.
5. Choose **Deploy**.

Step 5: Invoke your API

To invoke your API

1. From the main navigation pane, choose **Stage**.
2. Under **Stage details**, choose the copy icon to copy your API's invoke URL.

3.Enter the invoke URL in a web browser.

The full URL should look like `https://abcd123.execute-api.us-east-2.amazonaws.com/Prod`.

Your browser sends a `GET` request to the API.

4. Verify your API's response. You should see the text `"The API Gateway REST API console is great!"` in your browser.

Step 6: Clean up

To delete your REST API

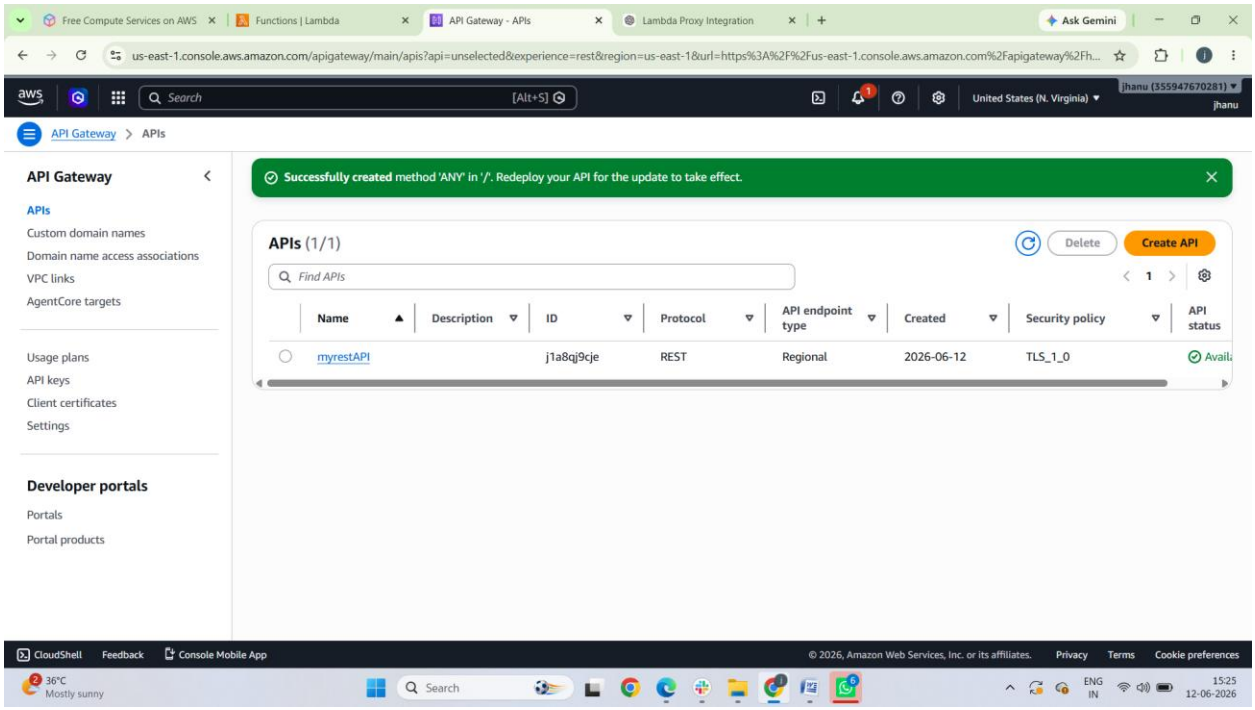
1. In the **Resources** pane, choose **API actions**, **Delete API**.
2. In the **Delete API** dialog box, enter **confirm**, and then choose **Delete**.

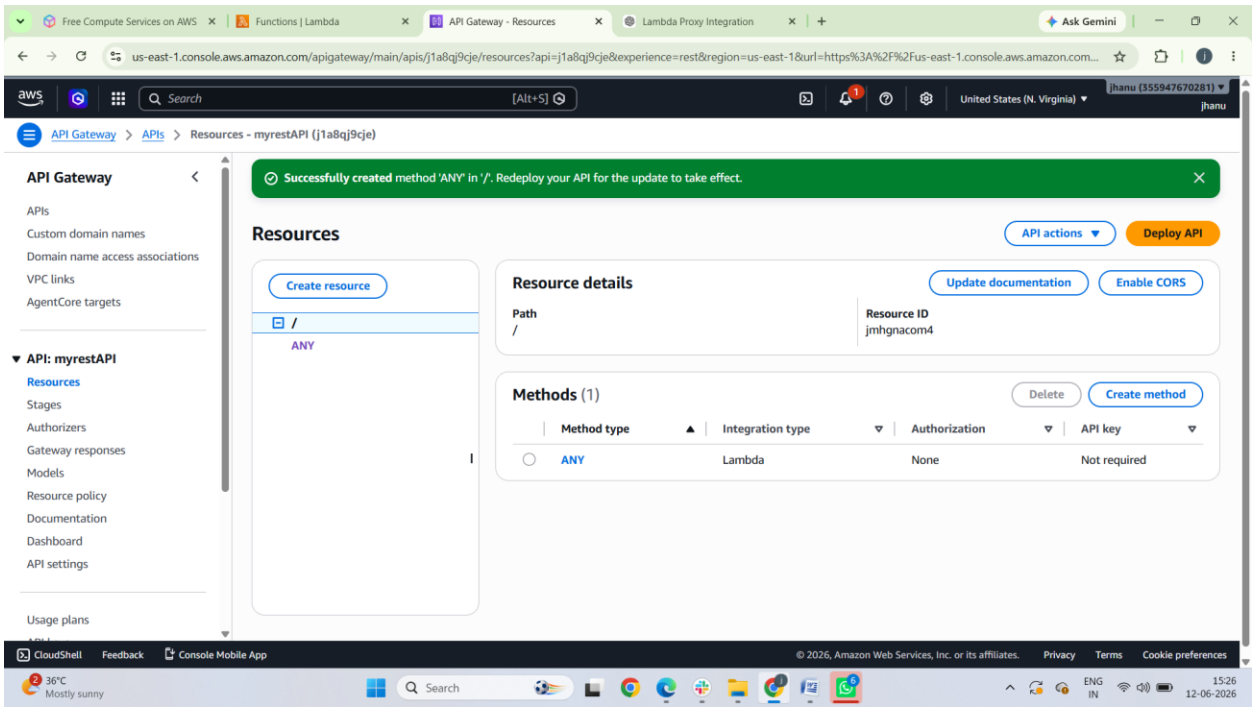
To delete your Lambda function

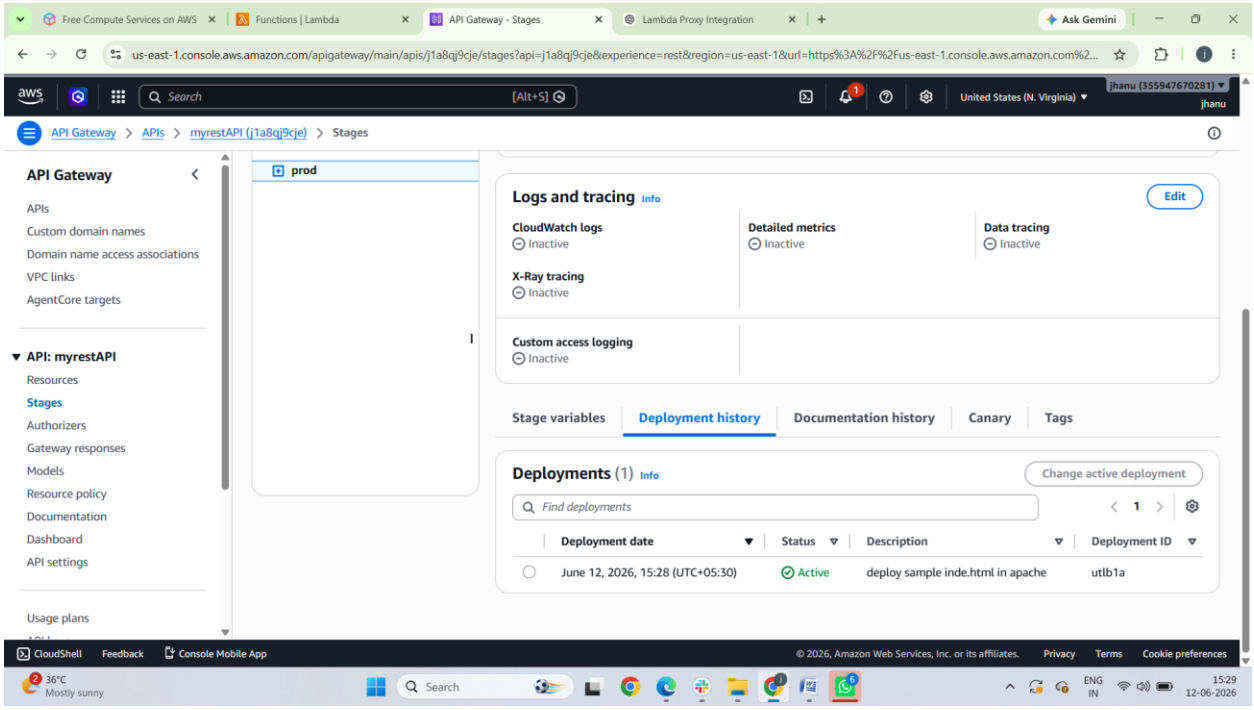
1. Sign in to the Lambda console at https://console.aws.amazon.com/lambda.
2. On the **Functions** page, select your function. Choose **Actions**, **Delete**.
3. In the **Delete 1 functions** dialog box, enter `delete`, and then choose **Delete**.

To delete your Lambda function's log group

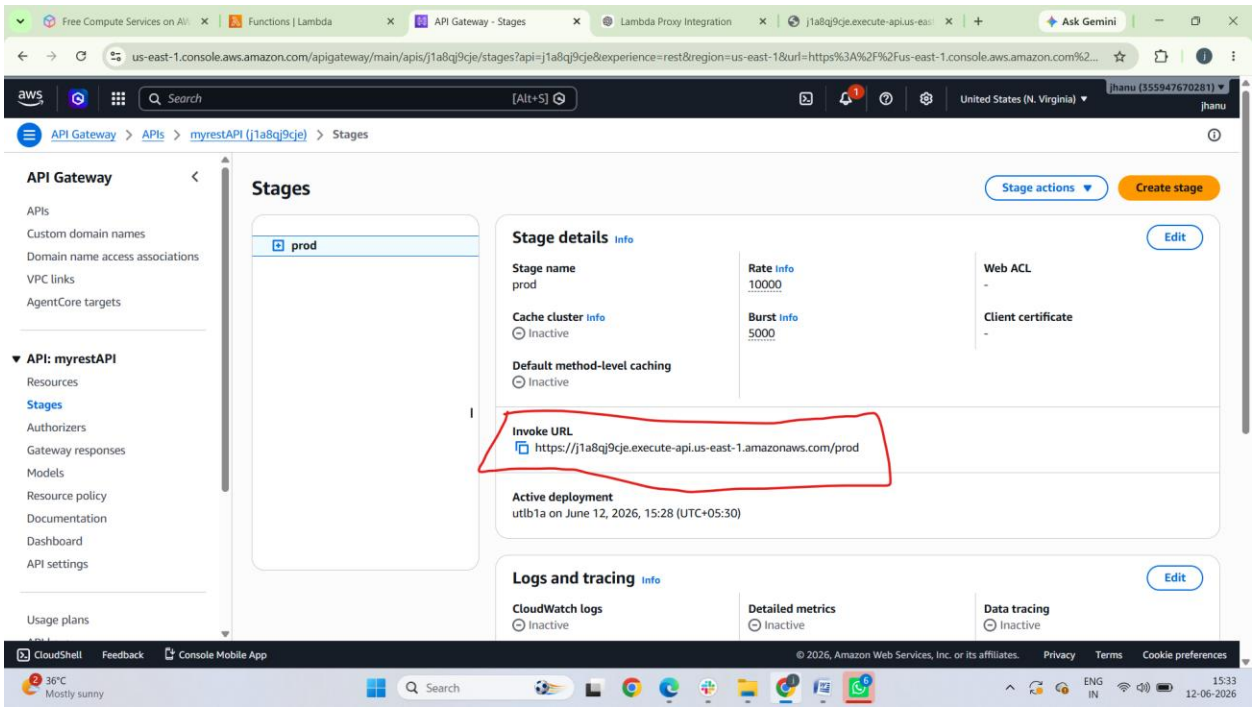
1. Open the Log groups page of the Amazon CloudWatch console.
2. On the **Log groups** page, select your function's log group (`/aws/lambda/my-function`). Then, for **Actions**, choose **Delete log group(s)**.
3. In the **Delete log group(s)** dialog box, choose **Delete**.

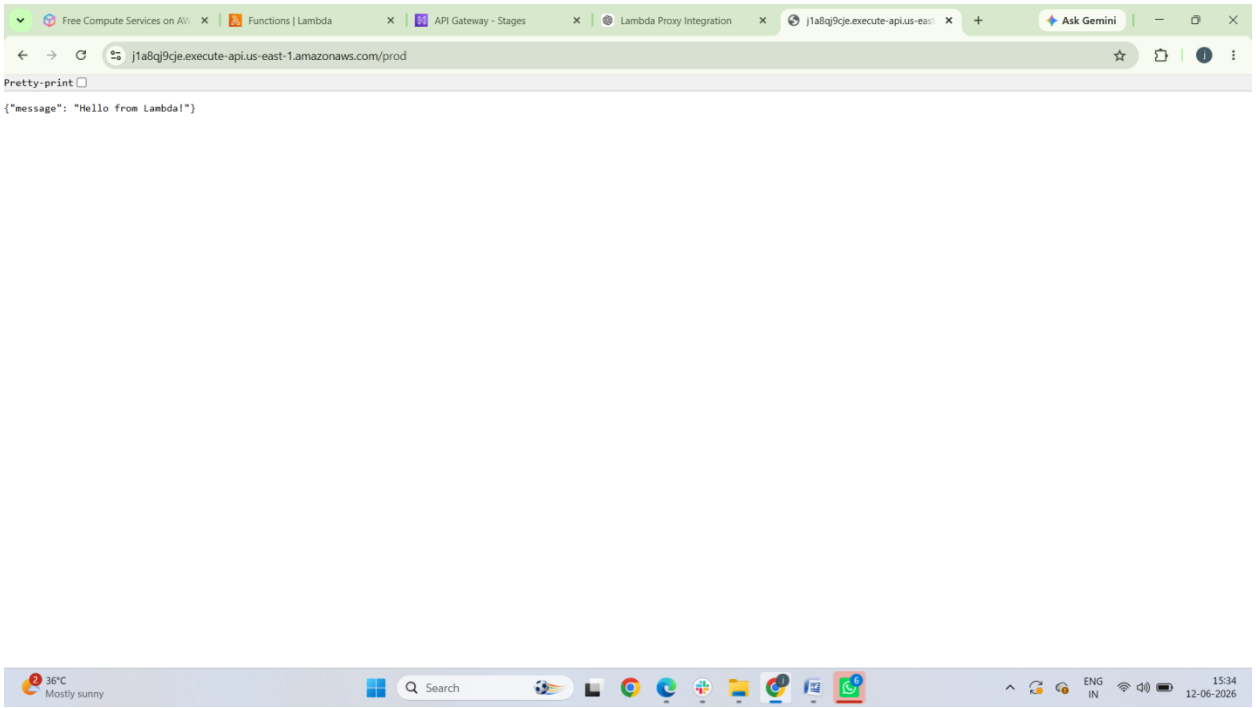


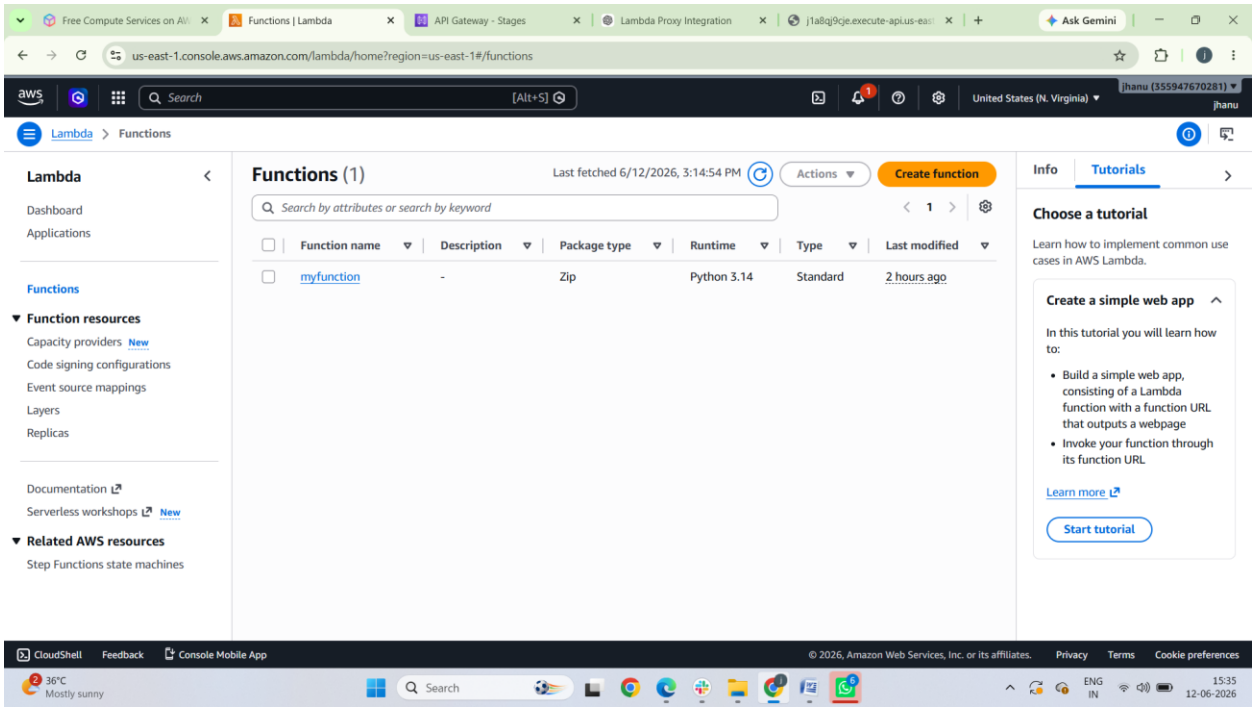

Free Compute Services on AWS
Functions | Lambda
API Gateway - Resources
Lambda Proxy Integration
Ask Gemini
us-east-1.console.aws.amazon.com/apigateway/main/apis/j1a8qj9cje/resources?api=j1a8qj9cje&experience=rest&region=us-east-1&url=https%3A%2F%2Fus-east-1.console.aws.amazon.com...
Search
[Alt+S]
United States (N. Virginia)
jhanu (355947670281)
jhanu
API Gateway > APIs > Resources - myrestAPI (j1a8qj9cje)
API Gateway
APIs
Custom domain names
Domain name access associations
VPC links
AgentCore targets
API: myrestAPI
Resources
Stages
Authorizers
Gateway responses
Models
Resource policy
Documentation
Dashboard
API settings
Usage plans
Successfully created method 'ANY' in '/'. Redeploy your API for the update to take effect.
Resources
API actions
Deploy API
Create resource
/
ANY
Resource details
Update documentation
Enable CORS
Path
/
Resource ID
jmhgnacom4
Methods (1)
Delete
Create method
Method type
Integration type
Authorization
API key
ANY
Lambda
None
Not required
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences
36°C
Mostly sunny
Search
ENG
IN
15:26
12-06-2026

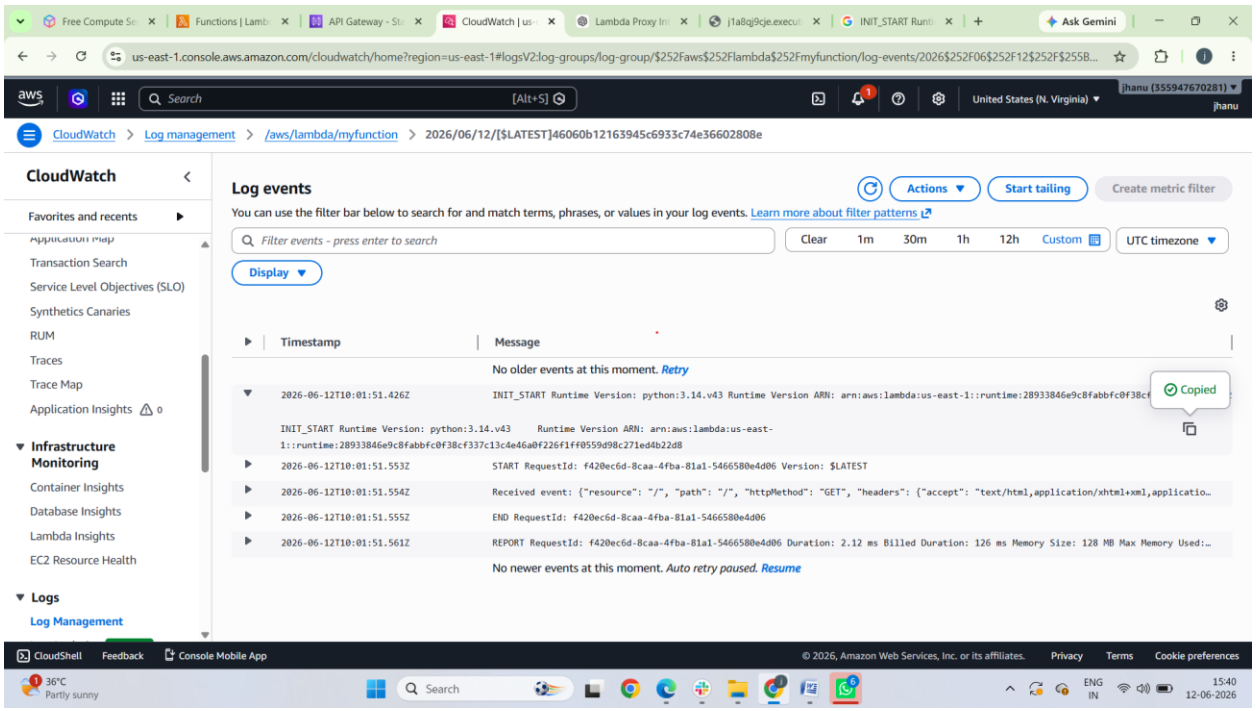

Free Compute Services on AWS
Functions | Lambda
API Gateway - Stages
Lambda Proxy Integration
Ask Gemini
us-east-1.console.aws.amazon.com/apigateway/main/apis/j1a8qj9cje/stages?api=j1a8qj9cje&experience=rest&region=us-east-1&url=https%3A%2F%2Fus-east-1.console.aws.amazon.com%2...
Search
[Alt+S]
United States (N. Virginia)
jhanu (355947670281)
jhanu
API Gateway > APIs > myrestAPI (j1a8qj9cje) > Stages
API Gateway
APIs
Custom domain names
Domain name access associations
VPC links
AgentCore targets
API: myrestAPI
Resources
Stages
Authorizers
Gateway responses
Models
Resource policy
Documentation
Dashboard
API settings
Usage plans
prod
Logs and tracing Info
Edit
CloudWatch logs
Inactive
Detailed metrics
Inactive
Data tracing
Inactive
X-Ray tracing
Inactive
Custom access logging
Inactive
Stage variables
Deployment history
Documentation history
Canary
Tags
Deployments (1) Info
Change active deployment
Find deployments
1
Deployment date
Status
Description
Deployment ID
June 12, 2026, 15:28 (UTC+05:30)
Active
deploy sample inde.html in apache
utlb1a
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences
36°C
Mostly sunny
Search
ENG
IN
15:29
12-06-2026

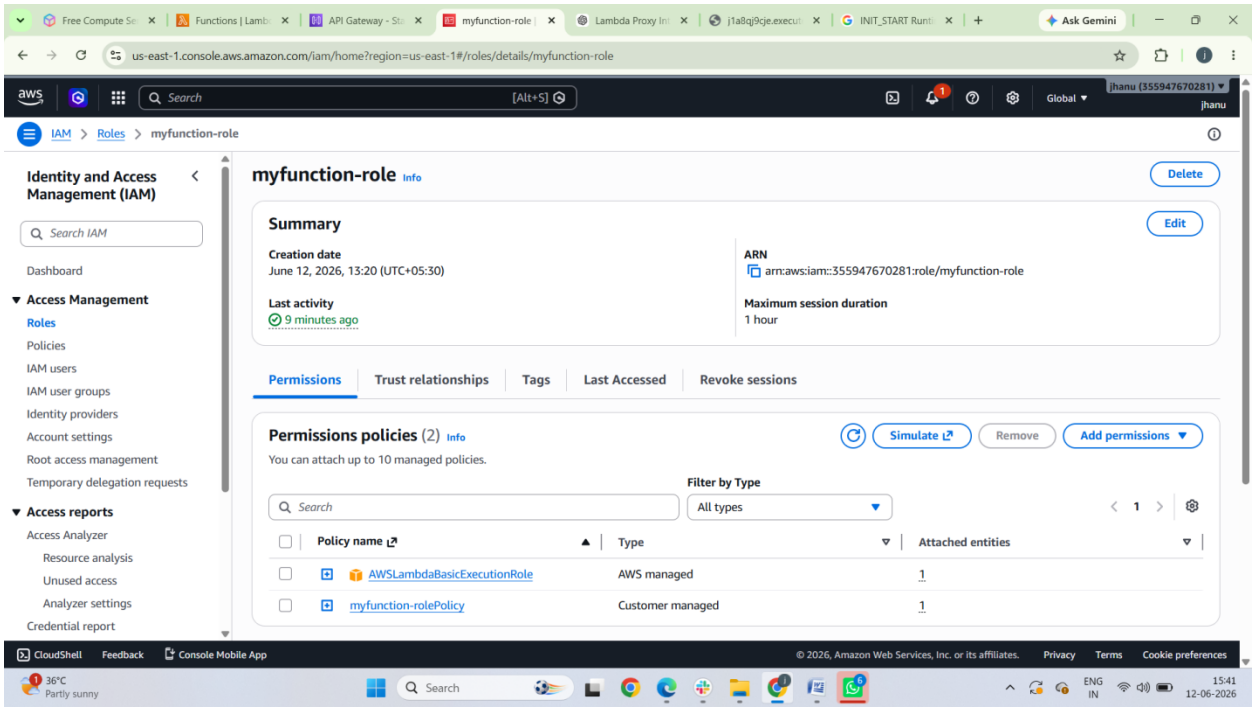
.

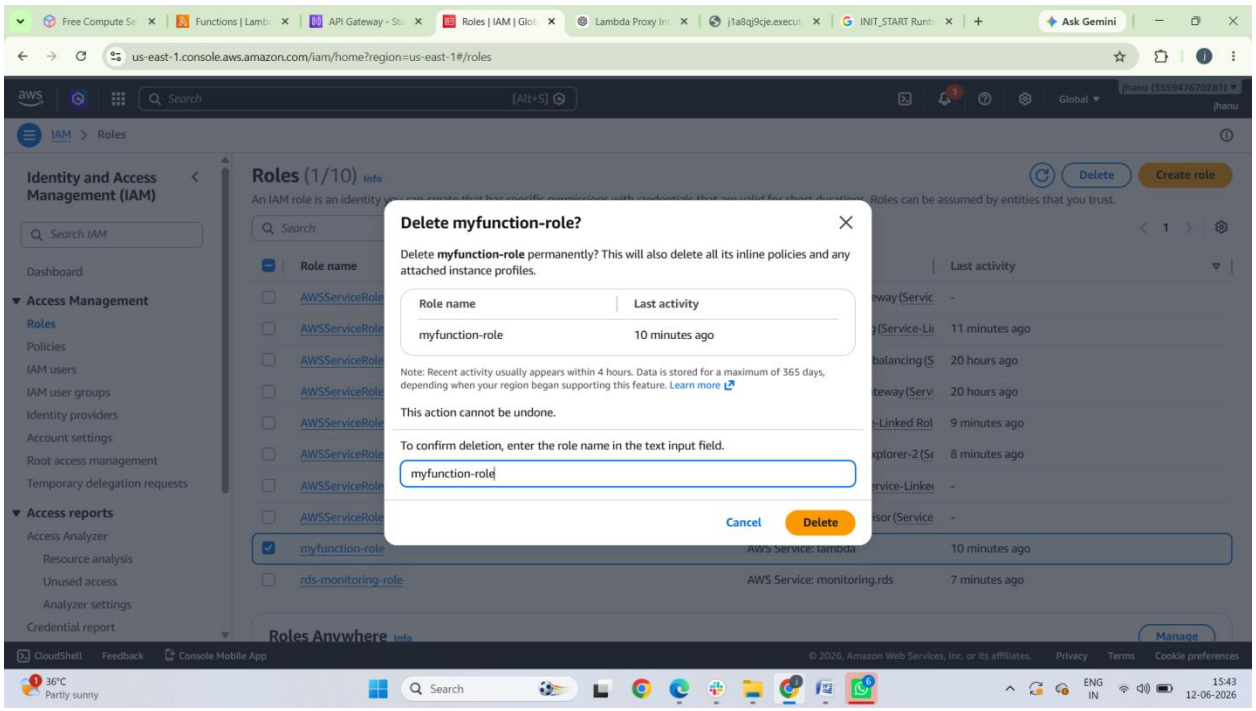

Free Compute Services on AW
Functions | Lambda
API Gateway - Stages
Lambda Proxy Integration
j1a8qj9cje.execute-api.us-east
Ask Gemini
us-east-1.console.aws.amazon.com/apigateway/main/apis/j1a8qj9cje/stages?api=j1a8qj9cje&experience=rest&region=us-east-1&url=https%3A%2F%2Fus-east-1.console.aws.amazon.com%2...
aws
Search
[Alt+S]
United States (N. Virginia)
jhanu (355947670281)
jhanu
API Gateway > APIs > myrestAPI (j1a8qj9cje) > Stages
API Gateway
APIs
Custom domain names
Domain name access associations
VPC links
AgentCore targets
API: myrestAPI
Resources
Stages
Authorizers
Gateway responses
Models
Resource policy
Documentation
Dashboard
API settings
Usage plans
Stages
Stage actions
Create stage
prod
Stage details Info
Edit
Stage name
prod
Rate Info
10000
Web ACL
-
Cache cluster Info
Inactive
Burst Info
5000
Client certificate
-
Default method-level caching
Inactive
Invoke URL
https://j1a8qj9cje.execute-api.us-east-1.amazonaws.com/prod
Active deployment
utlb1a on June 12, 2026, 15:28 (UTC+05:30)
Logs and tracing Info
Edit
CloudWatch logs
Inactive
Detailed metrics
Inactive
Data tracing
Inactive
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences
36°C
Mostly sunny
Search
ENG
IN
15:33
12-06-2026

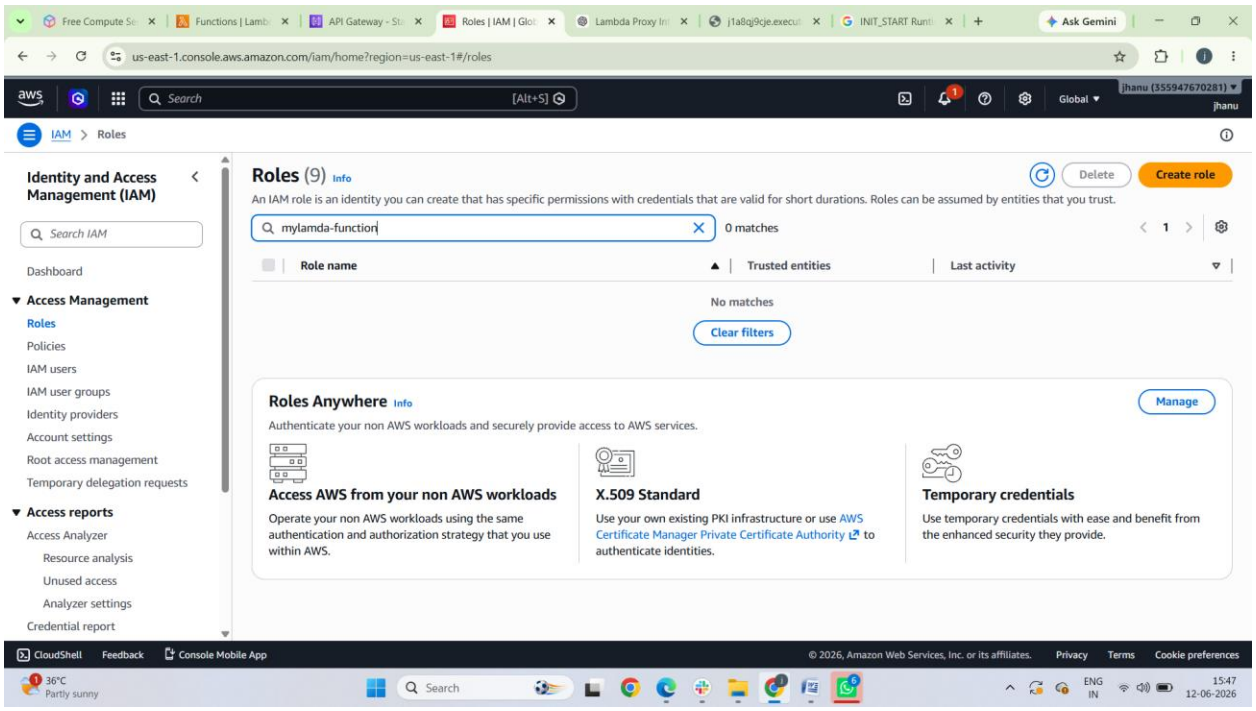

Free Compute Services on AW
Functions | Lambda
API Gateway - Stages
Lambda Proxy Integration
j1a8qj9cje.execute-api.us-east
Ask Gemini
j1a8qj9cje.execute-api.us-east-1.amazonaws.com/prod
Pretty-print
{"message": "Hello from Lambda!"}
36°C
Mostly sunny
Search
ENG
IN
15:34
12-06-2026


Free Compute Services on AW
Functions | Lambda
API Gateway - Stages
Lambda Proxy Integration
j1a8qj9cje.execute-api.us-east
Ask Gemini
us-east-1.console.aws.amazon.com/lambda/home?region=us-east-1#/functions
Search [Alt+S]
United States (N. Virginia)
jhanu (355947670281)
jhanu
Lambda > Functions
Lambda
Dashboard
Applications
Functions
Function resources
Capacity providers New
Code signing configurations
Event source mappings
Layers
Replicas
Documentation
Serverless workshops New
Related AWS resources
Step Functions state machines
Functions (1)
Last fetched 6/12/2026, 3:14:54 PM
Actions
Create function
Search by attributes or search by keyword
Function name
Description
Package type
Runtime
Type
Last modified
myfunction
-
Zip
Python 3.14
Standard
2 hours ago
Info
Tutorials
Choose a tutorial
Learn how to implement common use cases in AWS Lambda.
Create a simple web app
In this tutorial you will learn how to:
Build a simple web app, consisting of a Lambda function with a function URL that outputs a webpage
Invoke your function through its function URL
Learn more
Start tutorial
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences
36°C
Mostly sunny
Search
ENG
IN
15:35
12-06-2026


CloudWatch | us
INIT_START Runti
us-east-1.console.aws.amazon.com/cloudwatch/home?region=us-east-1#logsV2:log-groups/log-group/$252Faws$252Flambda$252Fmyfunction/log-events/2026$252F06$252F12$252F$255B...
CloudWatch > Log management > /aws/lambda/myfunction > 2026/06/12/[$LATEST]46060b12163945c6933c74e36602808e
CloudWatch
Favorites and recents
Transaction Search
Service Level Objectives (SLO)
Synthetics Canaries
RUM
Traces
Trace Map
Application Insights 0
Infrastructure Monitoring
Container Insights
Database Insights
Lambda Insights
EC2 Resource Health
Logs
Log Management
Log events
Actions
Start tailing
Create metric filter
You can use the filter bar below to search for and match terms, phrases, or values in your log events. Learn more about filter patterns
Filter events - press enter to search
Clear
1m
30m
1h
12h
Custom
UTC timezone
Display
Timestamp
Message
No older events at this moment. Retry
2026-06-12T10:01:51.426Z
INIT_START Runtime Version: python:3.14.v43 Runtime Version ARN: arn:aws:lambda:us-east-1::runtime:28933846e9c8fabbfc0f38cf337c13c4e46a0f226f1ff0559d98c271ed4b22d8
Copied
2026-06-12T10:01:51.553Z
START RequestId: f420ec6d-8caa-4fba-81a1-5466580e4d06 Version: $LATEST
2026-06-12T10:01:51.554Z
Received event: {"resource": "/", "path": "/", "httpMethod": "GET", "headers": {"accept": "text/html,application/xhtml+xml,applicatio…
2026-06-12T10:01:51.555Z
END RequestId: f420ec6d-8caa-4fba-81a1-5466580e4d06
2026-06-12T10:01:51.561Z
REPORT RequestId: f420ec6d-8caa-4fba-81a1-5466580e4d06 Duration: 2.12 ms Billed Duration: 126 ms Memory Size: 128 MB Max Memory Used:…
No newer events at this moment. Auto retry paused. Resume
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences
36°C
Partly sunny
Search
ENG
IN
15:40
12-06-2026


us-east-1.console.aws.amazon.com/iam/home?region=us-east-1#/roles/details/myfunction-role
IAM > Roles > myfunction-role
Identity and Access Management (IAM)
Search IAM
Dashboard
Access Management
Roles
Policies
IAM users
IAM user groups
Identity providers
Account settings
Root access management
Temporary delegation requests
Access reports
Access Analyzer
Resource analysis
Unused access
Analyzer settings
Credential report
myfunction-role Info
Delete
Summary
Edit
Creation date
June 12, 2026, 13:20 (UTC+05:30)
ARN
arn:aws:iam::355947670281:role/myfunction-role
Last activity
9 minutes ago
Maximum session duration
1 hour
Permissions
Trust relationships
Tags
Last Accessed
Revoke sessions
Permissions policies (2) Info
You can attach up to 10 managed policies.
Simulate
Remove
Add permissions
Filter by Type
Search
All types
Policy name
Type
Attached entities
AWSLambdaBasicExecutionRole
AWS managed
1
myfunction-rolePolicy
Customer managed
1
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


us-east-1.console.aws.amazon.com/iam/home?region=us-east-1#/roles
Roles (1/10) Info
Delete
Create role
Delete myfunction-role?
Delete myfunction-role permanently? This will also delete all its inline policies and any attached instance profiles.
Role name
Last activity
myfunction-role
10 minutes ago
Note: Recent activity usually appears within 4 hours. Data is stored for a maximum of 365 days, depending when your region began supporting this feature. Learn more
This action cannot be undone.
To confirm deletion, enter the role name in the text input field.
myfunction-role
Cancel
Delete
rds-monitoring-role
AWS Service: monitoring.rds
7 minutes ago
Roles Anywhere Info
Manage



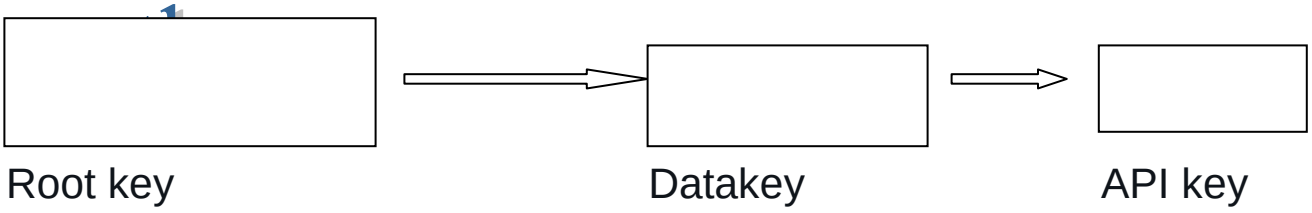
AWS Key Management Service

AWS Key Management Service (AWS KMS) is an AWS managed service that makes it easy for you to create and control the keys used to encrypt and sign your data

Use AWSKMS

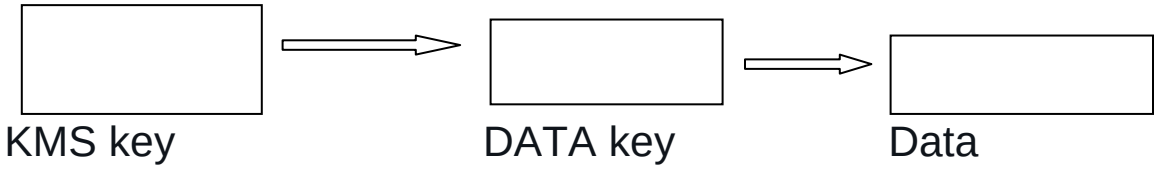
Amazon Web Services AWS Key Management Service (AWS KMS) **is used to create and control encryption keys for securing data in AWS services and your applications.**

If you encrypt your key, you need to protect its encryption key. Eventually, you must protect the highest level encryption key (known as a *root key*) in the hierarchy that protects your data. That's where AWS KMS comes in.



AWS KMS protects your root keys. KMS keys are created, managed, used, and deleted entirely within AWS KMS.

They never leave the service unencrypted. To use or manage your KMS keys, you call AWS KMS.



AWS KMS in AWS Regions

The AWS Regions in which AWS KMS is supported are listed in AWS Key Management Service Endpoints and Quotas

AWS KMS pricing

As with other AWS products, using AWS KMS does not require contracts or minimum purchases

AWS KMS service level agreement

AWS Key Management Service is backed by a service level agreement that defines our service availability policy

AWSKMS concepts

1. KMS Key (Customer Managed Key / AWS Managed Key)

A **KMS key** is the central cryptographic object in AWS KMS.

- Used to encrypt and decrypt data
- Has a unique Key ID and ARN
- Can be:
    - **AWS managed keys** (auto-created by AWS services)
    - **Customer managed keys** (you create and control)
    - **AWS owned keys** (fully managed, not visible to you)

2. Data Key (Envelope Encryption concept)

A **data key** is used for actual data encryption.

Flow:

1. KMS generates a data key
2. You encrypt data locally (faster)
3. You store:
    - Encrypted data
    - Encrypted data key

 This is called **envelope encryption** and is the standard pattern in AWS.

3. Key Policy

A **key policy** is the most important access control layer.

It defines:

- Who can use the key
- Which IAM roles/users can encrypt/decrypt
- Cross-account access rules

 Even if IAM allows access, the key policy must also allow it.

4. IAM Policy (Access Layer 2)

IAM policies define:

- Which identities can call KMS APIs

Example permissions:

- `kms:Encrypt`
- `kms:Decrypt`
- `kms:GenerateDataKey`

 IAM + Key Policy must BOTH allow access.

5. Grants

A **grant** is a temporary permission mechanism.

Used when:

- AWS services need short-term access to your key
- Example: S3, Lambda, EBS

 Grants reduce the need to constantly edit key policies.

6. Encryption Context

A set of **key-value pairs** used to add integrity checks.

Example:

```
{
  "Service": "S3",
  "User": "12345"
}
```

- Must match during decrypt
- Prevents ciphertext reuse attacks

7. Envelope Encryption

The most important real-world pattern.

Steps:

1. Request data key from KMS
2. KMS returns:
    - Plaintext data key
    - Encrypted data key
3. Use plaintext key locally to encrypt data
4. Store encrypted data + encrypted data key

 Improves performance and scalability.

8. Multi-Region Keys

Allows:

- Same key material replicated across regions
- Useful for global apps

9. Automatic Key Rotation

- Enables yearly rotation of key material
- Improves security hygiene
- Old versions remain decryptable

10. Audit Logging

All KMS usage is logged in:

- AWS CloudTrail

Tracks:

- Who used the key
- When
- What operation (encrypt/decrypt)

Accessing AWS Key Management Service

1. Before you access AWS KMS

You must have:

- An AWS account
- IAM permissions for KMS (e.g., `kms:Encrypt`, `kms:Decrypt`)
- A key policy that allows your IAM user/role

Without both **IAM + Key Policy**, access will fail.

2. Access via AWS Management Console

Steps:

1. Open AWS Console
2. Search for **KMS**
3. Go to **Key Management Service**
4. From here you can:
    - Create keys
    - Manage key policies
    - Enable/disable keys
    - View usage logs (via CloudTrail integration)

3. Access via AWS CLI

Configure CLI

```
aws configure
```

List keys

```
aws kms list-keys
```

Describe a key

```
aws kms describe-key --key-id <key-id>
```

Encrypt data

```
aws kms encrypt \
  --key-id <key-id> \
  --plaintext fileb://data.txt
```

Decrypt data

```
aws kms decrypt \
  --ciphertext-blob fileb://encrypted.bin
```

4. Access via SDK (Python example)

```
import boto3

kms = boto3.client("kms")

response = kms.list_keys()
print(response["Keys"])
```

Encrypt example:

```
resp = kms.encrypt(
    KeyId="alias/my-key",
    Plaintext=b"hello world"
)
```

```
print(resp["CiphertextBlob"])
```

5. Common ways AWS services access KMS

AWS services do NOT call KMS manually like users. Instead:

- **S3 (SSE-KMS)** encrypts objects using KMS keys
- **EBS volumes** are encrypted via KMS
- **Lambda** uses KMS for environment variables
- **RDS** uses KMS for database encryption

This is done using **IAM roles + KMS grants** automatically.

6. Common access errors

□ AccessDeniedException

- Missing IAM permission OR key policy mismatch

□ DisabledKeyException

- Key is disabled

□ IncorrectRegion

- KMS keys are region-specific

7. Key idea to remember

Access to AWS KMS always follows this rule:

IAM policy allows it + Key policy allows it = Access granted

Create a symmetric encryption KMS key

1. Sign in to the AWS Management Console and open the AWS Key Management Service (AWS KMS) console at https://console.aws.amazon.com/kms.
2. To change the AWS Region, use the Region selector in the upper-right corner of the page.

3. In the navigation pane, choose **Customer managed keys**.
4. Choose **Create key**.
5. To create a symmetric encryption KMS key, for **Key type** choose **Symmetric**.
6. In **Key usage**, the **Encrypt and decrypt** option is selected for you.
7. Choose **Next**.
8. Type an alias for the KMS key. The alias name cannot begin with `aws/`. The `aws/` prefix is reserved by Amazon Web Services to represent AWS managed keys in your account.
   Aliases are required when you create a KMS key in the AWS Management Console. They are optional when you use the CreateKey operation

   9.(Optional) Type a description for the KMS key.

   You can add a description now or update it any time unless the key state is `Pending Deletion` or `Pending Replica Deletion`. To add, change, or delete the description of an existing customer managed key, edit the description on the details page for the KMS key in the AWS Management Console or use the UpdateKeyDescription operation.

   10.(Optional) Type a tag key and an optional tag value. To add more than one tag to the KMS key, choose **Add tag**

   11.Choose **Next**.

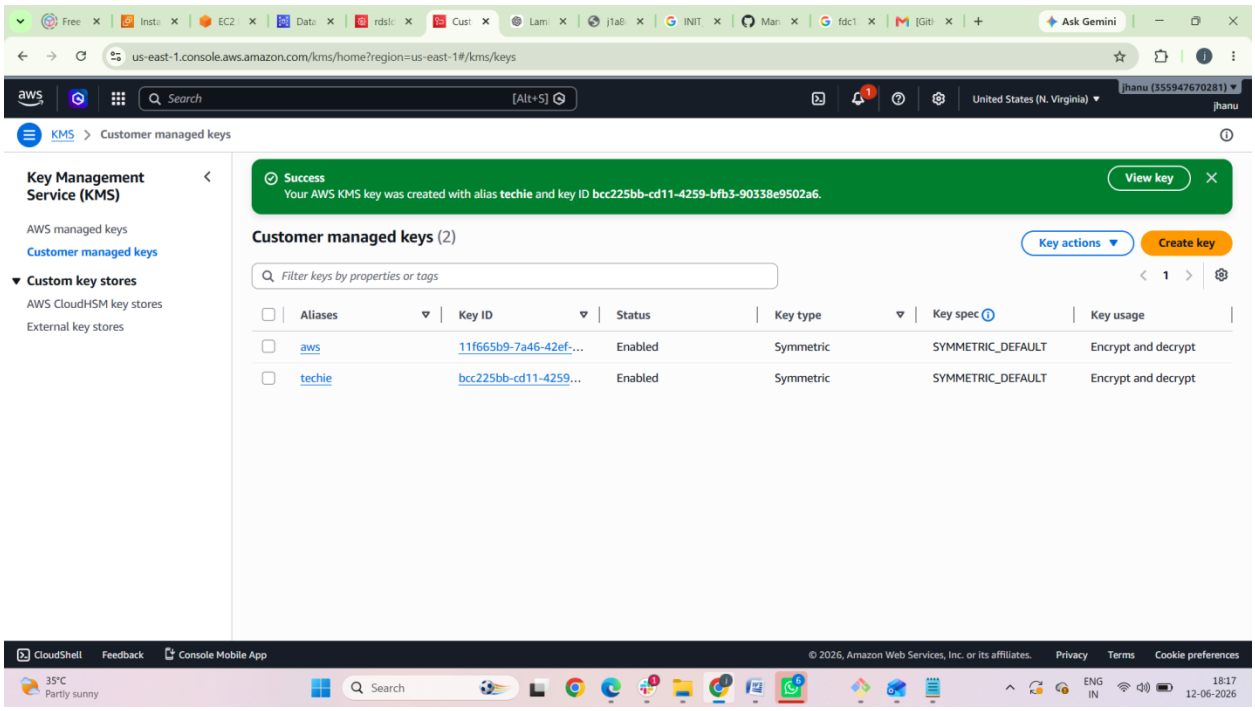
   12.Select the IAM users and roles that can administer the KMS key.

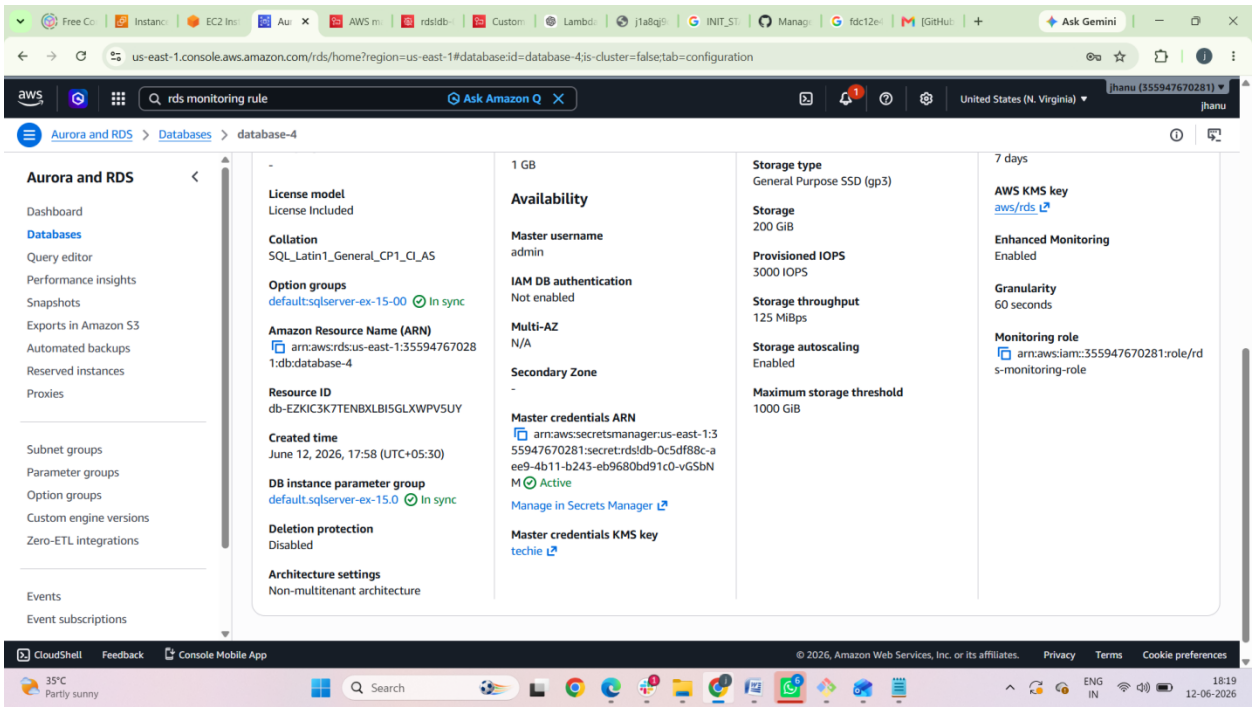
   13.(Optional) To prevent the selected IAM users and roles from deleting this KMS key, in the **Key deletion** section at the bottom of the page, clear the **Allow key administrators to delete this key** check box.

   14.Choose **Next**.

   15.Select the IAM users and roles that can use the key in cryptographic operations

   16. Optional) You can allow other AWS accounts to use this KMS key for cryptographic operations. To do so, in the **Other AWS accounts** section at the bottom of the page, choose **Add another AWS account** and enter the AWS account identification number of an external account. To add multiple external accounts, repeat this step
   17. Choose **Next**.
   18. Review the key policy statements for the key. To make changes to the key policy, select **Edit**.
   19. Choose **Next**.
   20. Review the key settings that you chose. You can still go back and change all settings.
   21. Choose **Finish** to create the KMS key.


us-east-1.console.aws.amazon.com/kms/home?region=us-east-1#/kms/keys
Search
[Alt+S]
United States (N. Virginia)
jhanu
KMS > Customer managed keys
Key Management Service (KMS)
AWS managed keys
Customer managed keys
Custom key stores
AWS CloudHSM key stores
External key stores
Success
Your AWS KMS key was created with alias techie and key ID bcc225bb-cd11-4259-bfb3-90338e9502a6.
View key
Customer managed keys (2)
Key actions
Create key
Filter keys by properties or tags
Aliases
Key ID
Status
Key type
Key spec
Key usage
aws
11f665b9-7a46-42ef-...
Enabled
Symmetric
SYMMETRIC_DEFAULT
Encrypt and decrypt
techie
bcc225bb-cd11-4259...
Enabled
Symmetric
SYMMETRIC_DEFAULT
Encrypt and decrypt
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences
35°C
Partly sunny
Search
ENG
IN
18:17
12-06-2026


us-east-1.console.aws.amazon.com/rds/home?region=us-east-1#database:id=database-4;is-cluster=false;tab=configuration
rds monitoring rule
Ask Amazon Q
United States (N. Virginia)
jhanu
Aurora and RDS > Databases > database-4
Aurora and RDS
Dashboard
Databases
Query editor
Performance insights
Snapshots
Exports in Amazon S3
Automated backups
Reserved instances
Proxies
Subnet groups
Parameter groups
Option groups
Custom engine versions
Zero-ETL integrations
Events
Event subscriptions
License model
License Included
Collation
SQL_Latin1_General_CP1_CI_AS
Option groups
default:sqlserver-ex-15-00
In sync
Amazon Resource Name (ARN)
arn:aws:rds:us-east-1:355947670281:db:database-4
Resource ID
db-EZKIC3K7TENBXLBI5GLXWPV5UY
Created time
June 12, 2026, 17:58 (UTC+05:30)
DB instance parameter group
default.sqlserver-ex-15.0
In sync
Deletion protection
Disabled
Architecture settings
Non-multitenant architecture
1 GB
Availability
Master username
admin
IAM DB authentication
Not enabled
Multi-AZ
N/A
Secondary Zone
Master credentials ARN
arn:aws:secretsmanager:us-east-1:355947670281:secret:rds!db-0c5df88c-aee9-4b11-b243-eb9680bd91c0-vGSbNM
Active
Manage in Secrets Manager
Master credentials KMS key
techie
Storage type
General Purpose SSD (gp3)
Storage
200 GiB
Provisioned IOPS
3000 IOPS
Storage throughput
125 MiBps
Storage autoscaling
Enabled
Maximum storage threshold
1000 GiB
7 days
AWS KMS key
aws/rds
Enhanced Monitoring
Enabled
Granularity
60 seconds
Monitoring role
arn:aws:iam::355947670281:role/rds-monitoring-role
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences
35°C
Partly sunny
Search
ENG
IN
18:19
12-06-2026